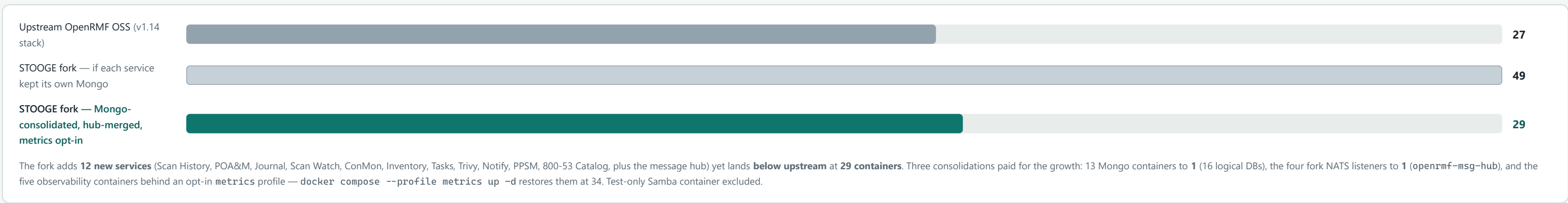


Architecture & Communication Ports / Protocols

Every box is a container on the internal `openrmf` Docker network. NGINX is the single host-exposed entry point; the NATS bus is the service backbone; every service owns exactly one **logical database** on a single consolidated MongoDB. All fork-built .NET services run on the **UBI8 (RHEL 8.10) base** with the DoD CA bundle. Teal marks fork additions/modifications.

29 containers default Æ 34 with metrics · 17 HTTP services · 7 NATS listeners · 1 Mongo container - 16 logical DBs · 1 host-exposed port (8080) · UBI8 fork services

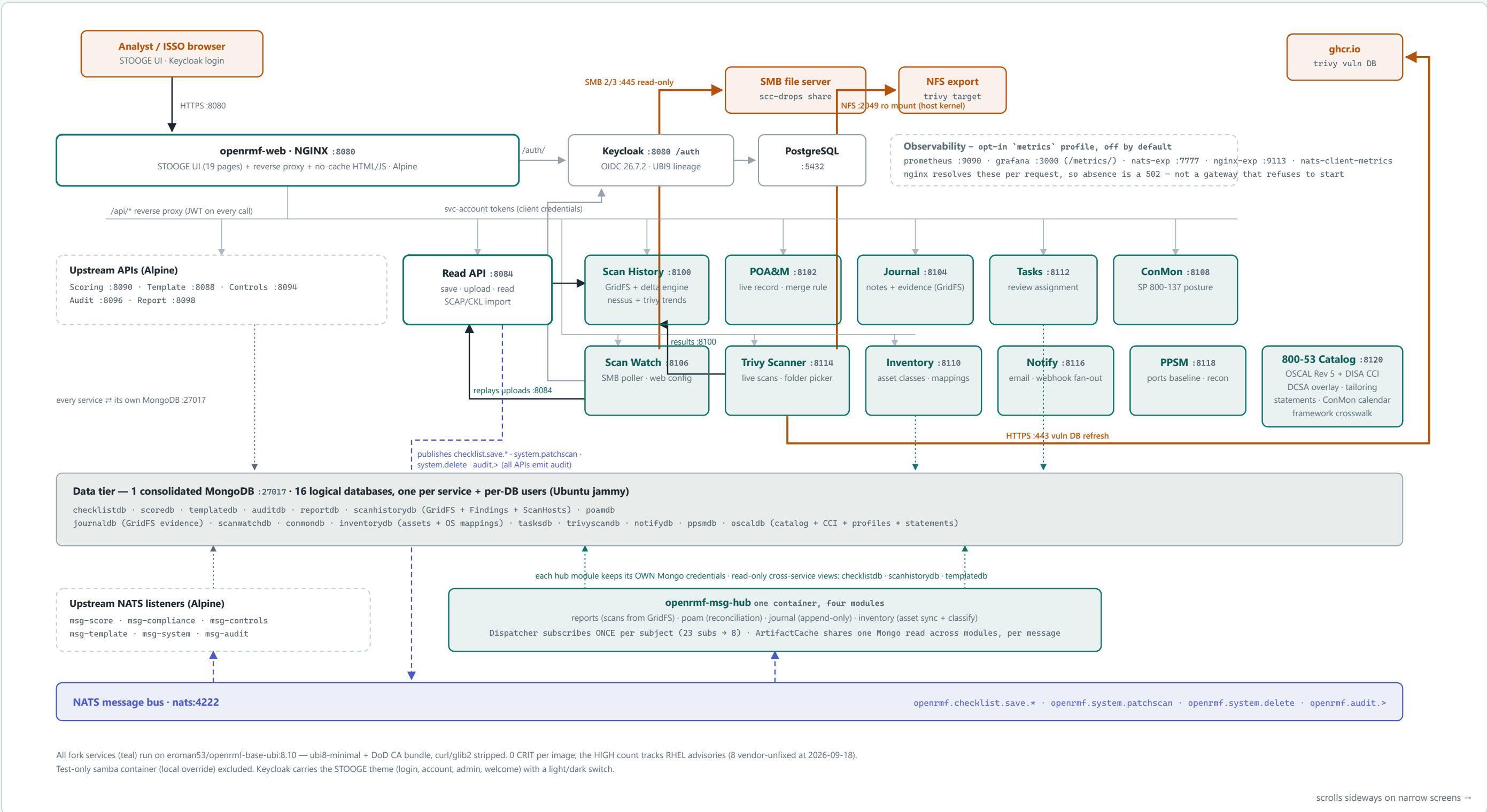
Container footprint vs. upstream OpenRMF



1 · Container architecture

☐ upstream (Alpine) ☒ fork-modified (UBI8) ☐ fork-new (UBI8) ☐ external

— HTTP / REST - - - NATS pub/hub MongoDB — SMB / NFS / registry (Boundary)



2 · Communication ports & protocols (PPSM view)

INBOUND (HOST BOUNDARY)					
1 port					
8080/tcp HTTP — the only host-published port; UI, all APIs, and OIDC ride through NGINX. TLS on 8443 is provisioned (commented) in the config.					
OUTBOUND (HOST BOUNDARY)					
3 flows					
445/tcp SMB (scan drops, read-only) · 2849/tcp NFS (trivy target, read-only, +111 for v3) · 443/tcp HTTPS to ghcr.io (Trivy DB; air-gap: pre-seed the cache volume)					
INTERNAL ONLY					
everything else					
All service, database, bus, auth, and metrics traffic stays on the internal <code>openrmf</code> Docker network — nothing below the gateway is host-published.					
SOURCE	DESTINATION	PORT	PROTOCOL / SERVICE	PURPOSE	
BOUNDARY-CROSSING					
Analyst / ISSO browser	openrmf-web (NGINX)	8080/tcp	HTTP (TLS 8443 available)	UI pages, all <code>/api/*</code> calls, OIDC login redirects	
openrmf-svc-scanwatch	SMB file server	445/tcp	SMB 2/3	Polls the scan drop share (read-only account)	
Docker host kernel	NFS server	2849/tcp (+111 v3)	NFS v4 (v3 opt.)	Read-only mount of the Trivy scan target	
openrmf-svc-trivy	ghcr.io	443/tcp	HTTPS / OCI	Trivy vulnerability DB refresh (cached in volume; skippable air-gapped)	
GATEWAY (NGINX REVERSE PROXY)					
NGINX	Keycloak	8080/tcp	HTTP / OIDC	<code>/auth/</code> — login, tokens, JWKS	
NGINX	Grafana (<code>metrics profile</code>)	3000/tcp	HTTP	<code>/metrics/</code> dashboards. Resolved per request via <code>resolver 127.0.0.11</code> — with the profile off the route returns 502 instead of preventing NGINX from starting.	
NGINX	Read API	8080/tcp	HTTP REST	<code>/api/read/</code> save · upload · read	
NGINX	Template / Scoring / Controls / Audit / Report APIs	8088 · 8090 · 8094 · 8096 · 8098/tcp	HTTP REST	Upstream API routes	
NGINX	Scan History / POA&M / Journal / Scan Watch / ConMon / Inventory / Tasks / Trivy Scanner / Notify / PPSM / 800-53 Catalog	8100 · 8102 · 8104 · 8106 · 8108 · 8110 · 8112 · 8114 · 8116 · 8118 · 8120/tcp	HTTP REST	Fork service routes <code>/api/scanhistory/</code> · <code>/api/poam/</code> · <code>/api/journal/</code> · <code>/api/scanmatch/</code> · <code>/api/conmon/</code> · <code>/api/inventory/</code> · <code>/api/tasks/</code> · <code>/api/trivyscan/</code> · <code>/api/notify/</code> · <code>/api/ppsm/</code> · <code>/api/oscal/</code>	
IDENTITY & AUTH					
Every API / service	Keycloak	8080/tcp	HTTP / OIDC	Issuer discovery + JWKS for JWT validation on every request	
Scan Watch · Trivy Scanner	Keycloak	8080/tcp	HTTP / OIDC token	Client-credentials tokens (service account <code>openrmf-scanwatch</code> , Editor+Reader)	
Keycloak	PostgreSQL	5432/tcp	PostgreSQL	Realm / user store	
SERVICE-TO-SERVICE HTTP					
Read API	Scan History API	8106/tcp	HTTP REST	Streams .nessus uploads into GridFS + delta engine	
Scan Watch	Read API	8080/tcp	HTTP REST	Replays dropped SCC XCCDF / CIL / Nessus files as authenticated uploads	
Trivy Scanner	Scan History API	8106/tcp	HTTP REST	Uploads Trivy JSON as <code>scanType</code> trivy snapshots	
MESSAGING BACKBONE					
Read API (+ every API for audit events)	NATS	4222/tcp	NATS pub	<code>openrmf.checklist.save.*</code> · <code>system.patchscan</code> · <code>system.delete</code> · <code>audit.*</code>	
7 msg listeners — 6 upstream (score · compliance · controls · template · system · audit) + <code>openrmf-msg-hub</code>	NATS	4222/tcp	NATS sub	Event-driven reconciliation, journaling, scoring, reporting. The hub subscribes once per subject and runs its four modules in a deterministic order, replacing four containers that each received the same three high-traffic subjects.	
DATA TIER					
Each service	Consolidated MongoDB — own logical DB + user	27017/tcp	MongoDB wire	One database per service, isolated by per-DB credentials — no shared writes	
msg-hub (reports · poam · journal · inventory modules, each with its own credentials) · ConMon · Inventory API · 800-53 Catalog	checklistdb · scanhistorydb · templatedb · poamdb · oscaldb	27017/tcp	MongoDB wire	Deliberate read-only cross-service views (findings, scan hosts, checklists, templates)	
OBSERVABILITY — OPT-IN METRICS PROFILE, ABSENT FROM A DEFAULT DEPLOYMENT					
Prometheus	Service <code>/metrics</code> endpoints · NATS exporter · NGINX exporter	service ports · 7777 · 9113/tcp	HTTP scrape	Metrics collection across every tier	
NATS exporter	NATS monitoring	8222/tcp	HTTP	<code>varz</code> · <code>connz</code> / <code>subz</code>	
Grafana	Prometheus	9090/tcp	HTTP / PromQL	Dashboards (served to users via NGINX <code>/metrics/</code>)	

Sources: `openrmf-decs/scripts/docker-compose.yml` · `nginx.conf`, regenerated **2026-09-20** for the **v0.9.0 beta** release. This revision folds in the four-lister merge into `openrmf-msg-hub`, the opt-in `metrics` profile (29 containers default / 34 with 0), Keycloak 26.7.2, and the DCSA overlay, ConMon calendar and framework crosswalk added to the 800-53 Catalog service. Service names, ports and profiles were read from `docker compose config --services` and the live `docker ps` rather than transcribed. At release every fork image scans **0 CRITICAL**; the 8 HIGH are vendor-unfixed and inherited from the RHEL 8.10 base. Keycloak carries 3 CRITICAL in jars bundled inside the upstream distribution, each one patch version behind its fix — accepted, and `trivy-scan.ps1` — Summary will keep falling on them by design.